

Team 157 - NLP Assignment 1: PS-3

Resume Analysis Pipeline

```
In [1]: # Install dependencies

%pip install nltk gensim hmmlearn scikit-learn pandas numpy tensorflow
import nltk
nltk.download("punkt_tab", quiet=True)
nltk.download('averaged_perceptron_tagger_eng')
```

Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: hmmlearn in /usr/local/lib/python3.12/dist-packages (0.3.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: scipy<=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open<=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.6.1)
Requirement already satisfied: threadpoolctl<=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil<=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz<=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata<=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: absl-py<=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse<=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers<=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta<=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang<=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum<=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf<=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six<=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor<=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions<=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt<=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.1)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.0)

Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)

Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.47.0)

Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)

Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.18)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2026.5.20)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.10.2)

Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (11.3.0)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.1.8)

Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.3)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (4.2.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (2.20.0)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.2)

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
```

Out[1]: True

```
In [2]: """
NLP Pipeline - Intelligent Student Resume Analysis
PS-3 Assignment: Resume Analysis & Internship Recommendation
"""

#!pip install nltk gensim hmmlearn scikit-learn pandas numpy tensorflow

import os
import re
```

```

import json
import warnings
import numpy as np
import pandas as pd
from collections import Counter

warnings.filterwarnings("ignore")

# -----
# STEP 0 - Install & Import
# -----
try:
    import nltk
    from nltk.corpus import stopwords
    from nltk.tokenize import word_tokenize, sent_tokenize
    from nltk.tag import pos_tag
    from nltk.chunk import RegexpParser
    nltk.download("punkt", quiet=True)
    nltk.download("averaged_perceptron_tagger", quiet=True)
    nltk.download("averaged_perceptron_tagger_eng", quiet=True)
    nltk.download("stopwords", quiet=True)
    nltk.download("maxent_ne_chunker", quiet=True)
    nltk.download("words", quiet=True)
except ImportError:
    raise ImportError("Run: pip install nltk")

try:
    from gensim.models import Word2Vec
except ImportError:
    raise ImportError("Run: pip install gensim")

try:
    from hmmlearn import hmm
except ImportError:
    raise ImportError("Run: pip install hmmlearn")

try:
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import (
        classification_report,
        accuracy_score,
        precision_recall_fscore_support,
        confusion_matrix
    )
    from sklearn.preprocessing import LabelEncoder
except ImportError:
    raise ImportError("Run: pip install scikit-learn")

try:
    import tensorflow as tf
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
    from tensorflow.keras.preprocessing.text import Tokenizer
    from tensorflow.keras.preprocessing.sequence import pad_sequences
    TF_AVAILABLE = True

```

```

except ImportError:
    TF_AVAILABLE = False
    print("[INFO] TensorFlow not found. Neural LM will use n-gram fallback.")

# -----
# DOMAIN CONFIG
# -----
INTERNSHIP_DOMAINS = {
    "Data Science": [
        "python", "pandas", "numpy", "matplotlib", "seaborn", "scikit", "sklearn",
        "statistics", "regression", "classification", "clustering", "sql", "tableau",
        "data analysis", "data visualization", "machine learning", "jupyter", "r programming"
    ],
    "Artificial Intelligence": [
        "tensorflow", "keras", "pytorch", "deep learning", "neural network", "nlp",
        "computer vision", "reinforcement learning", "transformer", "bert", "gpt",
        "opencv", "cnn", "rnn", "lstm", "ai", "artificial intelligence"
    ],
    "Web Development": [
        "html", "css", "javascript", "react", "angular", "vue", "nodejs", "django",
        "flask", "bootstrap", "php", "mongodb", "mysql", "restapi", "typescript",
        "frontend", "backend", "fullstack", "web", "jquery"
    ],
    "Mobile App Development": [
        "android", "ios", "flutter", "kotlin", "swift", "react native", "java",
        "mobile", "app development", "xml", "firebase", "xcode", "android studio",
        "dart", "objective-c"
    ],
    "Cyber Security": [
        "cybersecurity", "network security", "ethical hacking", "penetration testing",
        "firewall", "encryption", "cryptography", "kali linux", "metasploit",
        "vulnerability", "security audit", "wireshark", "nmap", "ceh", "cissp"
    ],
    "Cloud Computing": [
        "aws", "azure", "google cloud", "gcp", "docker", "kubernetes", "devops",
        "terraform", "ansible", "jenkins", "ci/cd", "microservices", "cloud",
        "lambda", "s3", "ec2", "serverless", "infrastructure"
    ],
    "Business Analytics": [
        "excel", "powerbi", "tableau", "business intelligence", "bi", "analytics",
        "kpi", "dashboard", "market research", "forecasting", "erp", "sap",
        "business analysis", "reporting", "data driven", "stakeholder"
    ]
}

SKILL_KEYWORDS = list(set(kw for keywords in INTERNSHIP_DOMAINS.values() for kw in keywords))

STANDARD_INTERNSHIP_DOMAINS = [
    "Data Science",
    "Web Development",
    "Mobile App Development",
    "Artificial Intelligence",
    "Cyber Security",
    "Cloud Computing",
    "Business Analytics"
]

```

```

CATEGORY_TO_DOMAIN = {
    "Frontend Developer": "Web Development",
    "Backend Developer": "Web Development",
    "Full Stack Developer": "Web Development",
    "Mobile App Developer (iOS/Android)": "Mobile App Development",
    "Machine Learning Engineer": "Artificial Intelligence",
    "Data Scientist": "Data Science",
    "Python Developer": "Data Science",
    "Cloud Engineer": "Cloud Computing",
    "Cyber Security Analyst": "Cyber Security"
}

# -----
# COMPONENT 1 – Preprocessing
# -----

class ResumePreprocessor:
    def __init__(self):
        self.stop_words = set(stopwords.words("english"))

    def clean(self, text: str) -> str:
        """Remove noise from resume text."""
        text = str(text).lower()
        text = re.sub(r"http\S+|www\S+", "", text)           # URLs
        text = re.sub(r"\S+@\S+", "", text)                 # emails
        text = re.sub(r"^[a-z0-9\s\+\#]", " ", text)        # special chars (keep + and # for C
        text = re.sub(r"\s+", " ", text).strip()
        return text

    def tokenize(self, text: str) -> list:
        return word_tokenize(text)

    def remove_stopwords(self, tokens: list) -> list:
        return [t for t in tokens if t not in self.stop_words and len(t) > 1]

    def process(self, text: str) -> dict:
        cleaned = self.clean(text)
        tokens = self.tokenize(cleaned)
        filtered = self.remove_stopwords(tokens)
        sentences = sent_tokenize(cleaned)
        return {
            "cleaned": cleaned,
            "tokens": tokens,
            "filtered_tokens": filtered,
            "sentences": sentences
        }

# -----
# COMPONENT 2 – POS Tagger + Skill Extractor
# -----

class POSTagger:
    def tag(self, tokens: list) -> list:
        """Return (word, POS_tag) pairs."""
        return pos_tag(tokens)

    def extract_skills(self, text: str) -> list:

```

```

        """Match resume text against known skill keywords."""
        text_lower = text.lower()
        found = []
        for skill in SKILL_KEYWORDS:
            if skill in text_lower:
                found.append(skill)
        return sorted(set(found))

    def extract_nouns_and_proper(self, tagged: list) -> list:
        """Pull out NN* (nouns) and NNP* (proper nouns) – likely tech/project terms."""
        return [word for word, tag in tagged if tag.startswith("NN")]

    def analyze(self, tokens: list, raw_text: str) -> dict:
        tagged = self.tag(tokens)
        tag_counts = Counter(tag for _, tag in tagged)
        skills = self.extract_skills(raw_text)
        nouns = self.extract_nouns_and_proper(tagged)
        return {
            "tagged_tokens": tagged[:30], # sample for display
            "tag_distribution": dict(tag_counts.most_common(10)),
            "skills_found": skills,
            "key_nouns": list(set(nouns))[:20]
        }

# -----
# COMPONENT 2A – Parsing
# -----

class ResumeParser:
    def __init__(self):
        self.grammar = """NP: {<JJ>*<NN.*>+}
VP: {<VB.*><NP|PP>+}"""
        self.parser = RegexpParser(self.grammar)

    def parse(self, tokens: list) -> dict:
        tagged = pos_tag(tokens)
        tree = self.parser.parse(tagged)
        phrases = {"NP": [], "VP": []}
        for subtree in tree.subtrees(filter=lambda t: t.label() in phrases):
            phrase = " ".join(word for word, _ in subtree.leaves())
            phrases[subtree.label()].append(phrase)
        return phrases

    def analyze(self, text: str) -> dict:
        tokens = word_tokenize(str(text).lower())
        return self.parse(tokens)

# -----
# COMPONENT 3 – Word2Vec Embeddings
# -----

class EmbeddingModel:
    def __init__(self, vector_size=100, window=5, min_count=1):
        self.vector_size = vector_size
        self.window = window
        self.min_count = min_count
        self.model = None

```

```

def train(self, corpus: list):
    """corpus = list of token lists (one per resume)."""
    self.model = Word2Vec(
        sentences=corpus,
        vector_size=self.vector_size,
        window=self.window,
        min_count=self.min_count,
        workers=4,
        epochs=10
    )
    print(f" [Embeddings] Trained Word2Vec on {len(corpus)} documents. Vocab size: {len(se

def get_resume_vector(self, tokens: list) -> np.ndarray:
    """Average word vectors to get a document-level vector."""
    vectors = []
    for token in tokens:
        if token in self.model.wv:
            vectors.append(self.model.wv[token])
    if vectors:
        return np.mean(vectors, axis=0)
    return np.zeros(self.vector_size)

def most_similar_to_resume(self, tokens: list, topn=5) -> list:
    """Find tech terms most similar to the resume's content."""
    resume_vec = self.get_resume_vector(tokens)
    if np.all(resume_vec == 0):
        return []
    try:
        sims = self.model.wv.similar_by_vector(resume_vec, topn=topn + 20)
        # Filter to only skill keywords
        skill_sims = [(w, round(float(s), 3)) for w, s in sims if w in SKILL_KEYWORDS]
        return skill_sims[:topn]
    except Exception:
        return []

# -----
# COMPONENT 4 – Neural Language Model (LSTM)
# -----

class NeuralLM:
    """
    Trains a simple character/word-level LSTM to model resume language patterns.
    Used to compute perplexity-like scores per domain.
    Falls back to n-gram model if TensorFlow is unavailable.
    """

    def __init__(self, vocab_size=3000, max_len=50):
        self.vocab_size = vocab_size
        self.max_len = max_len
        self.tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>") if TF_AVAILABLE else
        self.model = None
        self.ngram_counts = Counter()

    def _build_model(self):
        model = Sequential([
            Embedding(self.vocab_size, 64, input_length=self.max_len - 1),

```



```

        LSTM(128, return_sequences=False),
        Dropout(0.2),
        Dense(self.vocab_size, activation="softmax")
    ])
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model

def _prepare_sequences(self, texts: list):
    """Create (context → next_word) training pairs."""
    self.tokenizer.fit_on_texts(texts)
    sequences = self.tokenizer.texts_to_sequences(texts)
    X, y = [], []
    for seq in sequences:
        for i in range(1, len(seq)):
            n = seq[max(0, i - self.max_len + 1): i + 1]
            padded = [0] * (self.max_len - len(n)) + n
            X.append(padded[:-1])
            y.append(padded[-1])
    return np.array(X), np.array(y)

def train_ngram_fallback(self, texts: list):
    """Simple bigram LM as fallback."""
    for text in texts:
        words = text.split()
        for i in range(len(words) - 1):
            self.ngram_counts[(words[i], words[i+1])] += 1
    print(" [Neural LM] Trained bigram language model (TF fallback).")

def train(self, texts: list, epochs=3):
    if not TF_AVAILABLE:
        self.train_ngram_fallback(texts)
        return
    print(" [Neural LM] Preparing sequences...")
    X, y = self._prepare_sequences(texts)
    if len(X) == 0:
        print(" [Neural LM] Not enough data. Using fallback.")
        self.train_ngram_fallback(texts)
        return
    self.model = self._build_model()
    self.model.fit(X, y, epochs=epochs, batch_size=64, verbose=0)
    print(f" [Neural LM] LSTM trained on {len(X)} sequences.")

def predict_next_words(self, seed_text: str, n=5) -> list:
    """Predict next likely words given a seed phrase."""
    if not TF_AVAILABLE or self.model is None:
        # Fallback: most common next words
        words = seed_text.lower().split()
        if words:
            last = words[-1]
            candidates = [(w2, c) for (w1, w2), c in self.ngram_counts.items() if w1 == last]
            candidates.sort(key=lambda x: -x[1])
            return [w for w, _ in candidates[:n]]
        return []
    seq = self.tokenizer.texts_to_sequences([seed_text])
    padded = pad_sequences(seq, maxlen=self.max_len - 1, padding="pre")
    preds = self.model.predict(padded, verbose=0)[0]
    top_indices = np.argsort(preds)[-n:][::-1]

```

```

idx_to_word = {v: k for k, v in self.tokenizer.word_index.items()}
return [idx_to_word.get(i, "") for i in top_indices if i in idx_to_word]

# -----
# COMPONENT 5 – HMM for Sequence Modeling
# -----
class HMMDomainModel:
    """
    One Gaussian HMM per domain.
    Each HMM learns the distribution of Word2Vec vectors for resumes in that domain.
    At inference, we score the resume against each HMM and pick the best.
    """

    def __init__(self, n_components=3):
        self.n_components = n_components
        self.models = {}
        self.label_encoder = LabelEncoder()

    def train(self, X_dict: dict):
        """
        X_dict: { domain_label: [resume_vector, ...] }
        """
        for domain, vectors in X_dict.items():
            if len(vectors) < self.n_components:
                continue
            arr = np.array(vectors)
            model = hmm.GaussianHMM(
                n_components=self.n_components,
                covariance_type="diag",
                n_iter=50,
                random_state=42
            )
            try:
                model.fit(arr)
                self.models[domain] = model
            except Exception as e:
                print(f" [HMM] Could not train for {domain}: {e}")
        print(f" [HMM] Trained {len(self.models)} domain HMMs.")

    def predict(self, vector: np.ndarray) -> tuple:
        """Return (best_domain, score_dict)."""
        scores = {}
        v = vector.reshape(1, -1)
        for domain, model in self.models.items():
            try:
                score = model.score(v)
                scores[domain] = round(float(score), 3)
            except Exception:
                scores[domain] = float("-inf")
        if not scores:
            return "Unknown", {}
        best = max(scores, key=scores.get)
        return best, scores

# -----

```

```

# COMPONENT 6 - TF-IDF + Random Forest Classifier
# -----
class DomainClassifier:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(
            max_features=5000,
            ngram_range=(1, 2),
            stop_words="english"
        )
        self.clf = RandomForestClassifier(n_estimators=100, random_state=42)
        self.label_encoder = LabelEncoder()
        self.is_trained = False

    def train(self, texts: list, labels: list):
        X = self.vectorizer.fit_transform(texts)
        y = self.label_encoder.fit_transform(labels)
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42, stratify=y
        )
        self.clf.fit(X_train, y_train)
        y_pred = self.clf.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(
            y_test, y_pred, average="macro", zero_division=0
        )
        precision_weighted, recall_weighted, f1_weighted, _ = precision_recall_fscore_support(
            y_test, y_pred, average="weighted", zero_division=0
        )
        report = classification_report(
            y_test, y_pred,
            target_names=self.label_encoder.classes_,
            output_dict=True,
            zero_division=0
        )
        cm = confusion_matrix(y_test, y_pred)
        self.is_trained = True
        print(f" [Classifier] Accuracy: {acc:.2%}")
        print(f" [Classifier] Macro F1: {f1_macro:.2%}, Weighted F1: {f1_weighted:.2%}")
        return {
            "accuracy": round(acc, 4),
            "report": report,
            "classes": list(self.label_encoder.classes_),
            "confusion_matrix": cm.tolist(),
            "macro_precision": round(precision_macro, 4),
            "macro_recall": round(recall_macro, 4),
            "macro_f1": round(f1_macro, 4),
            "weighted_precision": round(precision_weighted, 4),
            "weighted_recall": round(recall_weighted, 4),
            "weighted_f1": round(f1_weighted, 4)
        }

    def predict(self, text: str) -> tuple:
        X = self.vectorizer.transform([text])
        pred_idx = self.clf.predict(X)[0]
        proba = self.clf.predict_proba(X)[0]
        domain = self.label_encoder.inverse_transform([pred_idx])[0]
        confidence = round(float(proba.max()), 4)

```

```

# Build top-3 probabilities
top3_idx = np.argsort(proba)[-3:][::-1]
top3 = [(self.label_encoder.inverse_transform([i])[0], round(float(proba[i]), 3)) for i in top3_idx]
return domain, confidence, top3

def keyword_fallback(self, text: str) -> tuple:
    """Pure keyword scoring – used when classifier isn't trained."""
    text_lower = text.lower()
    scores = {}
    for domain, keywords in INTERNSHIP_DOMAINS.items():
        score = sum(1 for kw in keywords if kw in text_lower)
        scores[domain] = score
    best = max(scores, key=scores.get)
    total = sum(scores.values()) or 1
    conf = round(scores[best] / total, 3)
    top3 = sorted(scores.items(), key=lambda x: -x[1])[0:3]
    top3 = [(d, round(s/total, 3)) for d, s in top3]
    return best, conf, top3

# -----
# MASTER PIPELINE
# -----

class ResumePipeline:
    def __init__(self):
        self.preprocessor = ResumePreprocessor()
        self.pos_tagger = POSTagger()
        self.embedding_model = EmbeddingModel()
        self.neural_lm = NeuralLM()
        self.hmm_model = HMMDomainModel()
        self.parser = ResumeParser()
        self.classifier = DomainClassifier()
        self.trained = False
        self.metrics = {}

    def normalize_category(self, category: str, text: str = "") -> str:
        category = str(category).strip()
        if category in CATEGORY_TO_DOMAIN:
            return CATEGORY_TO_DOMAIN[category]

        text_lower = str(text).lower()
        scores = {
            domain: sum(1 for kw in INTERNSHIP_DOMAINS.get(domain, []) if kw in text_lower)
            for domain in STANDARD_INTERNSHIP_DOMAINS
        }
        if scores:
            best = max(scores, key=scores.get)
            return best if scores[best] > 0 else "Unknown"
        return "Unknown"

    def fit(self, df: pd.DataFrame):
        """
        Train all components on a dataset.
        df must have columns: 'Resume' and 'Category'
        """
        print("\n" + "="*55)
        print(" TRAINING NLP PIPELINE")

```

```

print("="*55)

texts = df["Resume"].tolist()
original_labels = df["Category"].tolist()
labels = [self.normalize_category(label, text)
           for label, text in zip(original_labels, texts)]

# 1. Preprocess all resumes
print("\n[1/5] Preprocessing resumes...")
processed = [self.preprocessor.process(t) for t in texts]
cleaned_texts = [p["cleaned"] for p in processed]
all_tokens = [p["filtered_tokens"] for p in processed]

# 2. Train Word2Vec embeddings
print("[2/5] Training Word2Vec embeddings...")
self.embedding_model.train(all_tokens)

# 3. Build vectors for each resume (for HMM)
print("[3/5] Building resume vectors + training HMM...")
domain_vectors = {d: [] for d in set(labels)}
for tokens, label in zip(all_tokens, labels):
    vec = self.embedding_model.get_resume_vector(tokens)
    domain_vectors[label].append(vec)
self.hmm_model.train(domain_vectors)

# 4. Train Neural LM
print("[4/5] Training Neural Language Model...")
self.neural_lm.train(cleaned_texts[:500]) # cap for speed

# 5. Train RF classifier
print("[5/5] Training internship domain classifier...")
self.metrics = self.classifier.train(cleaned_texts, labels)

self.trained = True
print("\n✓ Pipeline training complete.\n")

def analyze(self, resume_text: str, original_category: str = "Unknown") -> dict:
    """Run full analysis on a single resume."""
    proc = self.preprocessor.process(resume_text)
    pos_results = self.pos_tagger.analyze(proc["tokens"], resume_text)
    parse_results = self.parser.analyze(resume_text)

    resume_vec = self.embedding_model.get_resume_vector(proc["filtered_tokens"])
    similar_skills = self.embedding_model.most_similar_to_resume(proc["filtered_tokens"])

    seed = " ".join(pos_results["skills_found"][:3]) if pos_results["skills_found"] else "d
    predicted_words = self.neural_lm.predict_next_words(seed, n=5)

    hmm_domain, hmm_scores = self.hmm_model.predict(resume_vec)

    if self.trained and self.classifier.is_trained:
        clf_domain, confidence, top3_domains = self.classifier.predict(proc["cleaned"])
    else:
        clf_domain, confidence, top3_domains = self.classifier.keyword_fallback(resume_text

    return {
        "resume_category": original_category,

```

```

        "recommended_internship_domain": clf_domain,
        "domain_confidence": confidence,
        "top_domain_candidates": top3_domains,
        "skills_extracted": pos_results["skills_found"],
        "key_nouns": pos_results["key_nouns"],
        "phrase_chunks": parse_results,
        "pos_tag_distribution": pos_results["tag_distribution"],
        "embedding_similar_skills": similar_skills,
        "neural_lm_predictions": predicted_words,
        "hmm_domain": hmm_domain,
        "hmm_scores": dict(sorted(hmm_scores.items(), key=lambda x: -x[1][:5])),
        "cleaned_text": proc["cleaned"],
        "token_count": len(proc["tokens"]),
        "filtered_token_count": len(proc["filtered_tokens"]),
        "sentence_count": len(proc["sentences"]),
        "performance_metrics": self.metrics
    }

# -----
# DATASET LOADER
# -----
def load_dataset(path: str = None) -> pd.DataFrame:
    """
    Load resume dataset.
    Expected columns: Category, Resume
    If no path is provided, this function attempts to load kaggle_resume_dataset.csv from the s
    """
    if path and os.path.exists(path):
        df = pd.read_csv(path)
        print(f"[Dataset] Loaded {len(df)} resumes from {path}")
        return df

    try:
        base_dir = os.path.dirname(os.path.abspath(__file__))
    except NameError:
        base_dir = os.getcwd()

    default_path = os.path.join(base_dir, "kaggle_resume_dataset.csv")
    if os.path.exists(default_path):
        df = pd.read_csv(default_path)
        print(f"[Dataset] Loaded {len(df)} resumes from {default_path}")
        return df

    raise FileNotFoundError(
        "Dataset not found. Please provide path to kaggle_resume_dataset.csv or place it next t
    )

def display_result(result: dict):
    print("\n" + "="*55)
    print(" OUTPUT FROM TEST DATASET")
    print("="*55)
    print(f"Extracted student skills: {' '.join(result.get('skills_extracted', [])) or 'None'}")
    print(f"Recommended internship domain: {result.get('recommended_internship_domain', 'Unknown')}")
    print(f"Resume category: {result.get('resume_category', 'Unknown')}")
    print(f"Domain confidence: {result.get('domain_confidence', 0.0):.2f}")

```

```

print("\nNLP analysis results:")
print(f" - Cleaned text: {result.get('cleaned_text', '')[:200]}{'...' if len(result.get('c
print(f" - Token count: {result.get('token_count')}}"")
print(f" - Filtered token count: {result.get('filtered_token_count')}}"")
print(f" - Sentence count: {result.get('sentence_count')}}"")
print(f" - POS tag distribution: {result.get('pos_tag_distribution')}}"")
print(f" - Key nouns: {result.get('key_nouns')}}"")
print(f" - Phrase chunks (NP): {result.get('phrase_chunks', {}).get('NP', [])[:10]}"")
print(f" - Phrase chunks (VP): {result.get('phrase_chunks', {}).get('VP', [])[:10]}"")
print(f" - Similar embedding skills: {result.get('embedding_similar_skills')}}"")
print(f" - Neural LM predictions: {result.get('neural_lm_predictions')}}"")
print(f" - HMM predicted domain: {result.get('hmm_domain')}}"")
print(f" - HMM scores: {result.get('hmm_scores')}}"")
print(f" - Top domain candidates: {result.get('top_domain_candidates')}}"")

print("\nPerformance metrics:")
metrics = result.get('performance_metrics', {})
print(f" - Classifier accuracy: {metrics.get('accuracy', 'N/A')}}"")
print(f" - Macro F1: {metrics.get('macro_f1', 'N/A')}}"")
print(f" - Weighted F1: {metrics.get('weighted_f1', 'N/A')}}"")
if isinstance(metrics.get('classes'), list) and isinstance(metrics.get('confusion_matrix'),
    print(" - Confusion matrix classes:")
    for cls, row in zip(metrics.get('classes'), metrics.get('confusion_matrix')):
        print(f"      {cls}: {row}")
if isinstance(metrics.get('report'), dict):
    print(f" - Classification report classes: {list(metrics.get('report', {}).keys())[:5]}"")
print(" " + "="*55 + "\n")

# -----
# MAIN – Run & Export Results
# -----

def run_pipeline(resume_text: str, dataset_path: str = None, original_category: str = "Unknown")
    df = load_dataset(dataset_path)
    pipeline = ResumePipeline()
    pipeline.fit(df)
    result = pipeline.analyze(resume_text, original_category=original_category)
    return result

if __name__ == "__main__":
    #Train the pipeline on the dataset
    df = load_dataset()
    pipeline = ResumePipeline()
    pipeline.fit(df)

    # Test the pipeline on the entire test dataset and display results for each resume
    try:
        base_dir = os.path.dirname(os.path.abspath(__file__))
    except NameError:
        base_dir = os.getcwd()
    df_test = pd.read_csv(os.path.join(base_dir, "kaggle_resume_dataset_test.csv"))

    for idx, test_row in df_test.iterrows():
        print(f"\n--- Test record {idx + 1} / {len(df_test)} ---")

```

```
result = pipeline.analyze(test_row["Resume"], original_category=test_row["Category"])  
display_result(result)
```


[Dataset] Loaded 394 resumes from /content/kaggle_resume_dataset.csv

```
=====
TRAINING NLP PIPELINE
=====
```

```
[1/5] Preprocessing resumes...
[2/5] Training Word2Vec embeddings...
[Embeddings] Trained Word2Vec on 394 documents. Vocab size: 1113
[3/5] Building resume vectors + training HMM...
[HMM] Trained 5 domain HMMs.
[4/5] Training Neural Language Model...
[Neural LM] Preparing sequences...
[Neural LM] LSTM trained on 43264 sequences.
[5/5] Training internship domain classifier...
[Classifier] Accuracy: 100.00%
[Classifier] Macro F1: 100.00%, Weighted F1: 100.00%
```

✓ Pipeline training complete.

--- Test record 1 / 9 ---

```
=====
OUTPUT FROM TEST DATASET
=====
```

```
Extracted student skills: aws, azure, backend, cloud, google cloud, mongodb, mysql, sql
Recommended internship domain: Web Development
Resume category: Data Scientist
Domain confidence: 0.57
```

NLP analysis results:

- Cleaned text: with a solid background in backend development i bring over 7 years of experience in building robust and scalable server side applications my expertise lies in designing and implementing restful apis ...
- Token count: 134
- Filtered token count: 89
- Sentence count: 1
- POS tag distribution: {'NN': 35, 'JJ': 23, 'IN': 17, 'NNS': 17, 'CC': 9, 'VBG': 7, 'VBP': 6, 'DT': 3, 'VBZ': 3, 'VB': 3}
- Key nouns: ['azure', 'business', 'experience', 'response', 'times', 'problem', 'development', 'requirements', 'database', 'frameworks', 'team', 'mongodb', 'applications', 'apis', 'queries', 'aws', 'cloud', 'player', 'teams', 'code']
- Phrase chunks (NP): ['solid background', 'backend development', 'i', 'years', 'experience', 'building robust', 'scalable server-side applications', 'expertise', 'restful apis', 'frameworks']
- Phrase chunks (VP): ['implementing restful apis', 'using frameworks', 'ensures fast response times', 'have extensive experience', 'deployed applications', 'tackle complex architectural challenges', 'optimize database queries', 'deliver solutions']
- Similar embedding skills: []
- Neural LM predictions: ['developer', 'technologies', 'development', 'app', 'with']
- HMM predicted domain: Web Development
- HMM scores: {'Web Development': 157.612, 'Data Science': 109.189, 'Mobile App Development': -313.795, 'Cloud Computing': -804.293, 'Artificial Intelligence': -1620.013}
- Top domain candidates: [(np.str_('Web Development'), 0.57), (np.str_('Data Science'), 0.15), (np.str_('Cloud Computing'), 0.13)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:

Artificial Intelligence: [13, 0, 0, 0, 0, 0]

Cloud Computing: [0, 16, 0, 0, 0, 0]

Cyber Security: [0, 0, 1, 0, 0, 0]

Data Science: [0, 0, 0, 29, 0, 0]

Mobile App Development: [0, 0, 0, 0, 13, 0]

Web Development: [0, 0, 0, 0, 0, 47]

- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

=====

--- Test record 2 / 9 ---

=====

OUTPUT FROM TEST DATASET

=====

Extracted student skills: backend, data analysis, django, flask, microservices, mongodb, mysql, numpy, pandas, python, sql, web

Recommended internship domain: Data Science

Resume category: Mobile App Developer (iOS/Android)

Domain confidence: 0.65

NLP analysis results:

- Cleaned text: as a python developer i leverage my expertise in the python programming language to build scalable and efficient backend solutions for diverse applications with over 4 years of experience i am profici...

- Token count: 142

- Filtered token count: 83

- Sentence count: 1

- POS tag distribution: {'NN': 37, 'IN': 16, 'NNS': 16, 'JJ': 12, 'CC': 10, 'VBP': 8, 'VB': 8, 'VBG': 7, 'DT': 6, 'TO': 6}

- Key nouns: ['experience', 'tasks', 'numpy', 'tools', 'automation', 'pandas', 'g', 'database', 'frameworks', 'mongodb', 'applications', 'apis', 'flask', 'knowledge', 'data', 'foundation', 'developer', 'field', 'python', 'manipulation']

- Phrase chunks (NP): ['python developer', 'i leverage', 'expertise', 'python', 'language', 'efficient backend solutions', 'diverse applications', 'years', 'experience', 'i']

- Phrase chunks (VP): ['programming language', 'am proficient', 'develop restful apis', 'retrieve data', 'using libraries', 'derive valuable insights', 'develop scripts', 'streamline repetitive tasks', 'improve workflow efficiency', 'evolving field']

- Similar embedding skills: [('php', 0.978)]

- Neural LM predictions: ['and', 'with', 'developer', 'skilled', 'java']

- HMM predicted domain: Data Science

- HMM scores: {'Data Science': 181.982, 'Web Development': 69.271, 'Mobile App Development': -710.062, 'Artificial Intelligence': -1029.663, 'Cloud Computing': -1487.227}

- Top domain candidates: [(np.str_('Data Science'), 0.65), (np.str_('Web Development'), 0.21), (np.str_('Artificial Intelligence'), 0.06)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:

```

Artificial Intelligence: [13, 0, 0, 0, 0, 0]
Cloud Computing: [0, 16, 0, 0, 0, 0]
Cyber Security: [0, 0, 1, 0, 0, 0]
Data Science: [0, 0, 0, 29, 0, 0]
Mobile App Development: [0, 0, 0, 0, 13, 0]
Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]
=====

```

--- Test record 3 / 9 ---

```

=====
OUTPUT FROM TEST DATASET
=====

```

```

Extracted student skills: ai, bi, classification, clustering, data visualization, deep learning, machine learning, matplotlib, python, pytorch, regression, scikit, seaborn, stakeholder, tableau, tensorflow
Recommended internship domain: Data Science
Resume category: Data Scientist
Domain confidence: 0.71

```

NLP analysis results:

```

- Cleaned text: with a background in data science i possess a unique blend of analytical skills programming expertise and domain knowledge to tackle complex data driven problems i have over 6 years of experience work...
- Token count: 164
- Filtered token count: 101
- Sentence count: 1
- POS tag distribution: {'NN': 40, 'IN': 20, 'NNS': 20, 'JJ': 19, 'VBG': 12, 'CC': 12, 'VBP': 8, 'VB': 8, 'DT': 7, 'TO': 7}
- Key nouns: ['learning', 'experience', 'tools', 'visualization', 'stakeholders', 'innovation', 'knowledge', 'classification', 'data', 'tableau', 'seaborn', 'languages', 'learn', 'regression', 'skills', 'python', 'environments', 'insights', 'algorithms', 'decision']
- Phrase chunks (NP): ['background', 'data science', 'unique blend', 'analytical skills', 'expertise', 'knowledge', 'complex data-driven problems', 'i', 'years', 'experience']
- Phrase chunks (VP): ['programming expertise', 'domain knowledge', 'tackle complex data-driven problems', 'conducting statistical analysis', 'learning models', 'extract meaningful insights', 'informed decision-making', 'programming languages', 'learning algorithms', 'using tools']
- Similar embedding skills: [('matplotlib', 0.956), ('statistics', 0.942), ('numpy', 0.935)]
- Neural LM predictions: ['developer', 'engineer', 'app', 'with', 'backend']
- HMM predicted domain: Data Science
- HMM scores: {'Data Science': -57.075, 'Web Development': -362.952, 'Artificial Intelligence': -447.752, 'Mobile App Development': -1488.384, 'Cloud Computing': -2161.196}
- Top domain candidates: [(np.str_('Data Science'), 0.71), (np.str_('Artificial Intelligence'), 0.15), (np.str_('Cyber Security'), 0.05)]

```

Performance metrics:

```

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
  Artificial Intelligence: [13, 0, 0, 0, 0, 0]
  Cloud Computing: [0, 16, 0, 0, 0, 0]
  Cyber Security: [0, 0, 1, 0, 0, 0]
  Data Science: [0, 0, 0, 29, 0, 0]

```

```

Mobile App Development: [0, 0, 0, 0, 13, 0]
Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]
=====

```

--- Test record 4 / 9 ---

```

=====
OUTPUT FROM TEST DATASET
=====

```

```

Extracted student skills: ai, aws, azure, bi, cloud, encryption, gcp, infrastructure, network security, terraform
Recommended internship domain: Cloud Computing
Resume category: Cloud Engineer
Domain confidence: 0.97

```

NLP analysis results:

```

- Cleaned text: skilled cloud engineer with a focus on building secure and scalable cloud environments proficient in aws azure and gcp with expertise in services such as iam cloud security and cloud networking skills...
- Token count: 102
- Filtered token count: 72
- Sentence count: 1
- POS tag distribution: {'NN': 32, 'NNS': 15, 'IN': 14, 'JJ': 10, 'CC': 9, 'VBG': 9, 'VBN': 3, 'DT': 2, 'RB': 2, 'JJ': 2}
- Key nouns: ['azure', 'learning', 'certifications', 'networking', 'tools', 'protection', 'automation', 'compliance', 'gcp', 'security', 'training', 'network', 'encryption', 'threats', 'data', 'cloud', 'policies', 'disaster', 'environments', 'terraform']
- Phrase chunks (NP): ['skilled cloud', 'focus', 'building secure', 'scalable cloud environments', 'proficient', 'aws', 'azure', 'gcp', 'expertise', 'services']
- Phrase chunks (VP): ['using tools', 'led projects', 'implementing identity', 'configuring encryption', 'designing disaster recovery strategies', 'conducting compliance audits', 'providing training']
- Similar embedding skills: []
- Neural LM predictions: ['developer', 'and', 'engineer', 'with', 'app']
- HMM predicted domain: Cloud Computing
- HMM scores: {'Cloud Computing': 213.514, 'Web Development': -400.348, 'Data Science': -445.978, 'Mobile App Development': -703.909, 'Artificial Intelligence': -1235.076}
- Top domain candidates: [(np.str_('Cloud Computing'), 0.97), (np.str_('Web Development'), 0.01), (np.str_('Artificial Intelligence'), 0.01)]

```

Performance metrics:

```

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
  Artificial Intelligence: [13, 0, 0, 0, 0, 0]
  Cloud Computing: [0, 16, 0, 0, 0, 0]
  Cyber Security: [0, 0, 1, 0, 0, 0]
  Data Science: [0, 0, 0, 29, 0, 0]
  Mobile App Development: [0, 0, 0, 0, 13, 0]
  Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]
=====

```

--- Test record 5 / 9 ---

=====

OUTPUT FROM TEST DATASET

=====

Extracted student skills: css, flask, html, java, javascript, mongodb, mysql, react, sql, vue, web

Recommended internship domain: Web Development

Resume category: Full Stack Developer

Domain confidence: 0.94

NLP analysis results:

- Cleaned text: experienced full stack developer with expertise in developing end to end web applications proficient in front end technologies such as html css javascript react vue and back end technologies such as n...

- Token count: 97

- Filtered token count: 73

- Sentence count: 1

- POS tag distribution: {'NN': 35, 'JJ': 13, 'NNS': 13, 'IN': 9, 'CC': 8, 'VBG': 7, 'VBN': 3, 'TO': 2, 'VBD': 2, 'RB': 2}

- Key nouns: ['learning', 'development', 'database', 'mongodb', 'applications', 'apis', 'javascript', 'flask', 'trends', 'stack', 'react', 'authorization', 'end', 'developer', 'authentication', 'web', 'party', 'postgresql', 'systems', 'performance']

- Phrase chunks (NP): ['full stack developer', 'expertise', 'end-to-end web applications', 'proficient', 'front-end technologies', 'html', 'css', 'javascript', 'vue', 'back-end technologies']

- Phrase chunks (VP): ['experienced full stack developer', 'developing end-to-end web applications', 'using sql', 'led projects', 'building e-commerce platforms', 'implementing authentication', 'integrating third-party apis', 'optimizing application performance']

- Similar embedding skills: []

- Neural LM predictions: ['developer', 'engineer', 'with', 'scientist', 'frontend']

- HMM predicted domain: Web Development

- HMM scores: {'Web Development': 32.346, 'Data Science': -63.557, 'Mobile App Development': -569.486, 'Cloud Computing': -1199.412, 'Artificial Intelligence': -1399.128}

- Top domain candidates: [(np.str_('Web Development'), 0.94), (np.str_('Data Science'), 0.04), (np.str_('Mobile App Development'), 0.01)]

Performance metrics:

- Classifier accuracy: 1.0

- Macro F1: 1.0

- Weighted F1: 1.0

- Confusion matrix classes:

Artificial Intelligence: [13, 0, 0, 0, 0, 0]

Cloud Computing: [0, 16, 0, 0, 0, 0]

Cyber Security: [0, 0, 1, 0, 0, 0]

Data Science: [0, 0, 0, 29, 0, 0]

Mobile App Development: [0, 0, 0, 0, 13, 0]

Web Development: [0, 0, 0, 0, 0, 47]

- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

=====

--- Test record 6 / 9 ---

```
=====
OUTPUT FROM TEST DATASET
=====
```

Extracted student skills: angular, bi, css, excel, frontend, html, java, javascript, react, vue, web
 Recommended internship domain: Web Development
 Resume category: Cloud Engineer
 Domain confidence: 0.88

NLP analysis results:

- Cleaned text: experienced frontend developer with a passion for crafting elegant and responsive user interfaces with over 6 years of hands on experience i specialize in building interactive web applications using h...
- Token count: 144
- Filtered token count: 97
- Sentence count: 1
- POS tag distribution: {'NN': 37, 'NNS': 23, 'IN': 19, 'JJ': 14, 'CC': 11, 'VBP': 9, 'VBG': 7, 'RB': 5, 'DT': 4, 'VBN': 3}
- Key nouns: ['experience', 'tools', 'expectations', 'development', 'assets', 'frameworks', 'applications', 'javascript', 'components', 'devices', 'trends', 'angular', 'data', 'react', 'frontend', 'developer', 'quality', 'experiences', 'skills', 'user']
- Phrase chunks (NP): ['experienced frontend developer', 'passion', 'responsive user interface s', 'years', 'hands-on experience', 'building', 'interactive web applications', 'html', 'css', 'javascript']
- Phrase chunks (VP): ['using html', 'frontend frameworks', 'translating mockups', 'ensuring seamless user experiences', 'manage complex application states', 'optimize performance', 'deliver cutting-edge solutions', 'exceed client expectations']
- Similar embedding skills: []
- Neural LM predictions: ['developer', 'engineer', 'with', 'app', 'scientist']
- HMM predicted domain: Web Development
- HMM scores: {'Web Development': 171.39, 'Data Science': 105.478, 'Mobile App Development': -495.677, 'Cloud Computing': -1482.136, 'Artificial Intelligence': -1712.731}
- Top domain candidates: [(np.str_('Web Development'), 0.88), (np.str_('Data Science'), 0.04), (np.str_('Artificial Intelligence'), 0.03)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
 - Artificial Intelligence: [13, 0, 0, 0, 0, 0]
 - Cloud Computing: [0, 16, 0, 0, 0, 0]
 - Cyber Security: [0, 0, 1, 0, 0, 0]
 - Data Science: [0, 0, 0, 29, 0, 0]
 - Mobile App Development: [0, 0, 0, 0, 13, 0]
 - Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

```
-----
--- Test record 7 / 9 ---
-----
```

```
=====
OUTPUT FROM TEST DATASET
=====
```

Extracted student skills: ai, bi, cloud, erp, machine learning, network security, python, pytorc

h, scikit, tensorflow
 Recommended internship domain: Artificial Intelligence
 Resume category: Machine Learning Engineer
 Domain confidence: 0.81

NLP analysis results:

- Cleaned text: dynamic machine learning engineer with a focus on developing scalable and interpretable machine learning models proficient in python with experience in libraries such as scikit learn tensorflow and py...
- Token count: 99
- Filtered token count: 72
- Sentence count: 1
- POS tag distribution: {'NN': 35, 'NNS': 16, 'IN': 15, 'JJ': 8, 'VBG': 8, 'CC': 7, 'VBD': 2, 'RB': 2, 'TO': 2, 'DT': 1}
- Key nouns: ['learning', 'experience', 'explainability', 'security', 'network', 'subscription', 'evaluation', 'data', 'prediction', 'cloud', 'experts', 'python', 'tensorflow', 'systems', 'model', 'conferences', 'detection', 'platforms', 'models', 'scikit']
- Phrase chunks (NP): ['dynamic machine', 'engineer', 'focus', 'interpretable machine learning models', 'proficient', 'python', 'experience', 'libraries', 'scikit-learn', 'tensorflow']
- Phrase chunks (VP): ['learning engineer', 'led projects', 'developing customer churn prediction models', 'building sentiment analysis models', 'implementing anomaly detection systems', 'including model explainability']
- Similar embedding skills: [('regression', 0.99), ('clustering', 0.968)]
- Neural LM predictions: ['developer', 'engineer', 'scientist', 'with', 'frontend']
- HMM predicted domain: Artificial Intelligence
- HMM scores: {'Artificial Intelligence': 233.337, 'Data Science': -480.11, 'Web Development': -783.65, 'Mobile App Development': -1934.959, 'Cloud Computing': -2202.871}
- Top domain candidates: [(np.str_('Artificial Intelligence'), 0.81), (np.str_('Data Science'), 0.17), (np.str_('Cloud Computing'), 0.02)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
 - Artificial Intelligence: [13, 0, 0, 0, 0, 0]
 - Cloud Computing: [0, 16, 0, 0, 0, 0]
 - Cyber Security: [0, 0, 1, 0, 0, 0]
 - Data Science: [0, 0, 0, 29, 0, 0]
 - Mobile App Development: [0, 0, 0, 0, 13, 0]
 - Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

=====

--- Test record 8 / 9 ---

=====

OUTPUT FROM TEST DATASET

=====

Extracted student skills: android, app development, bi, flutter, ios, java, kotlin, mobile, objective-c, swift
 Recommended internship domain: Mobile App Development
 Resume category: Mobile App Developer
 Domain confidence: 0.89

NLP analysis results:

- Cleaned text: dynamic mobile app developer with a diverse skill set in ios and android development proficient in swift objective c java and kotlin with experience in frameworks such as swiftui android jetpack and f...
- Token count: 93
- Filtered token count: 68
- Sentence count: 1
- POS tag distribution: {'NN': 25, 'JJ': 16, 'NNS': 13, 'IN': 11, 'VBG': 9, 'CC': 8, 'VBN': 3, 'DT': 2, 'VBD': 2, 'RB': 2}
- Key nouns: ['delivery', 'experience', 'capabilities', 'development', 'frameworks', 'apis', 'trends', 'apps', 'kotlin', 'developer', 'java', 'food', 'app', 'c', 'design', 'streaming', 'courses', 'platforms', 'platform', 'technologies']
- Phrase chunks (NP): ['dynamic mobile app developer', 'diverse skill', 'ios', 'android development', 'proficient', 'swift', 'java', 'kotlin', 'experience', 'frameworks']
- Phrase chunks (VP): ['led projects', 'developing food delivery apps', 'building educational platforms', 'implementing video streaming services', 'implementing offline capabilities']
- Similar embedding skills: [('firebase', 0.97)]
- Neural LM predictions: ['developer', 'engineer', 'with', 'frontend', 'machine']
- HMM predicted domain: Mobile App Development
- HMM scores: {'Mobile App Development': 96.37, 'Web Development': -85.521, 'Data Science': -123.687, 'Cloud Computing': -933.448, 'Artificial Intelligence': -1503.502}
- Top domain candidates: [(np.str_('Mobile App Development'), 0.89), (np.str_('Web Development'), 0.09), (np.str_('Data Science'), 0.01)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
 - Artificial Intelligence: [13, 0, 0, 0, 0, 0]
 - Cloud Computing: [0, 16, 0, 0, 0, 0]
 - Cyber Security: [0, 0, 1, 0, 0, 0]
 - Data Science: [0, 0, 0, 29, 0, 0]
 - Mobile App Development: [0, 0, 0, 0, 13, 0]
 - Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

=====

--- Test record 9 / 9 ---

=====

OUTPUT FROM TEST DATASET

=====

Extracted student skills: ai, encryption, ethical hacking, firewall, network security, penetration testing

Recommended internship domain: Web Development

Resume category: Cyber Security Analyst

Domain confidence: 0.34

NLP analysis results:

- Cleaned text: a dynamic cyber security analyst with expertise in network security and ethical hacking well trained in penetration testing firewall configuration and encryption
- Token count: 22
- Filtered token count: 16
- Sentence count: 1

- POS tag distribution: {'NN': 9, 'JJ': 4, 'IN': 3, 'CC': 2, 'DT': 1, 'RB': 1, 'VBN': 1, 'VBG': 1}
- Key nouns: ['hacking', 'penetration', 'security', 'network', 'expertise', 'encryption', 'analyst', 'configuration']
- Phrase chunks (NP): ['dynamic cyber security analyst', 'expertise', 'network security', 'ethical hacking', 'penetration testing', 'firewall configuration', 'encryption']
- Phrase chunks (VP): []
- Similar embedding skills: [('vulnerability', 0.974)]
- Neural LM predictions: ['developer', 'scientist', 'engineer', 'data', 'a']
- HMM predicted domain: Web Development
- HMM scores: {'Web Development': -180.587, 'Data Science': -243.457, 'Mobile App Development': -761.91, 'Cloud Computing': -839.781, 'Artificial Intelligence': -1305.785}
- Top domain candidates: [(np.str_('Web Development'), 0.34), (np.str_('Cyber Security'), 0.31), (np.str_('Data Science'), 0.13)]

Performance metrics:

- Classifier accuracy: 1.0
- Macro F1: 1.0
- Weighted F1: 1.0
- Confusion matrix classes:
 - Artificial Intelligence: [13, 0, 0, 0, 0, 0]
 - Cloud Computing: [0, 16, 0, 0, 0, 0]
 - Cyber Security: [0, 0, 1, 0, 0, 0]
 - Data Science: [0, 0, 0, 29, 0, 0]
 - Mobile App Development: [0, 0, 0, 0, 13, 0]
 - Web Development: [0, 0, 0, 0, 0, 47]
- Classification report classes: [np.str_('Artificial Intelligence'), np.str_('Cloud Computing'), np.str_('Cyber Security'), np.str_('Data Science'), np.str_('Mobile App Development')]

=====

BITS LAB Execution:

```

1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023
3024
3025
3026
3027
3028
3029
3030
3031
3032
3033
3034
3035
3036
3037
3038
3039
3040
3041
3042
3043
3044
3045
3046
3047
3048
3049
3050
3051
3052
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128
3129
3130
3131
3132
3133
3134
3135
3136
3137
3138
3139
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161
3162
3163
3164
3165
3166
3167
3168
3169
3170
3171
3172
3173
3174
3175
3176
3177
3178
3179
3180
3181
3182
3183
3184
3185
3186
3187
3188
3189
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
3200
3201
3202
3203
3204
3205
3206
3207
3208
3209
3210
3211
3212
3213
3214
3215
3216
3217
3218
3219
3220
3221
3222
3223
3224
3225
3226
3227
3228
3229
3230
3231
3232
3233
3234
3235
3236
3237
3238
3239
3240
3241
3242
3243
3244
3245
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256
3257
3258
3259
3260
3261
3262
3263
3264
3265
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
327
```